

BÁO CÁO CUỐI KỲ

THỰC TẬP CƠ SỞ



ĐỀ TÀI : WEBSITE TUYỂN DỤNG VIỆC LÀM



Đỗ Bình Minh - B23DCCN542



I. LÝ DO CHỌN ĐỀ TÀI

Thị trường lao động ngày càng phát triển, nhu cầu tìm việc và tuyển dụng rất lớn nhưng việc kết nối giữa ứng viên và nhà tuyển dụng vẫn còn rời rạc. Em muốn xây dựng một nền tảng đơn giản, dễ dùng để giải quyết bài toán đó — đồng thời áp dụng các công nghệ mới đang được thị trường ưa chuộng

Mục tiêu:

- Ứng viên tìm kiếm, xem và ứng tuyển việc làm dễ dàng
- Nhà tuyển dụng đăng tin và quản lý hồ sơ ứng viên
- Hệ thống phân quyền rõ ràng giữa 2 vai trò



II. CÁC CHỨC NĂNG CHÍNH

Ứng viên:

- Đăng ký / đăng nhập tài khoản
- Xem danh sách công ty, tin tuyển dụng
- Tìm kiếm việc theo từ khóa và thành phố
- Nộp CV ứng tuyển, theo dõi lịch sử đã nộp
- Cập nhật hồ sơ cá nhân



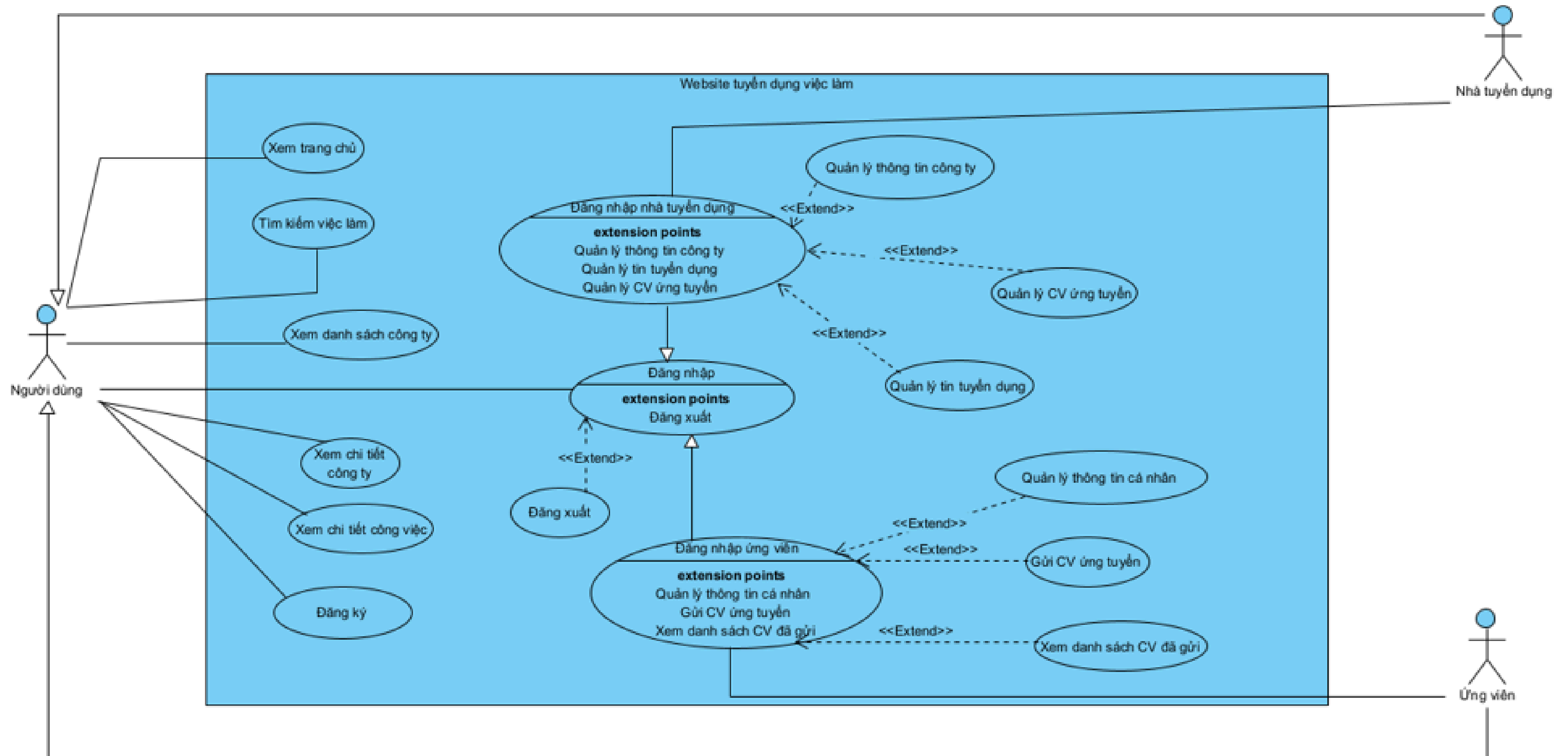
II. CÁC CHỨC NĂNG CHÍNH

Nhà tuyển dụng:

- Đăng ký / đăng nhập tài khoản doanh nghiệp
- Đăng, chỉnh sửa, xóa tin tuyển dụng
- Xem danh sách CV ứng viên đã nộp
- Cập nhật thông tin hồ sơ công ty



III. SƠ ĐỒ USECASE TỔNG QUÁT



IV. CÁC CÔNG NGHỆ SỬ DỤNG

Frontend

- **Next.js** — Framework React do Vercel phát triển, hỗ trợ render phía server (SSR) và routing tự động theo cấu trúc thư mục thông qua App Router.
- **Tailwind CSS** — Thư viện CSS theo hướng utility-first, giúp xây dựng giao diện nhanh mà không cần viết CSS thủ công.
- **TypeScript** — Phiên bản mở rộng của JavaScript, bổ sung kiểu dữ liệu tĩnh giúp phát hiện lỗi sớm và code dễ bảo trì hơn.



IV. CÁC CÔNG NGHỆ SỬ DỤNG

Backend

- **Node.js + Express.js** — Node.js là môi trường chạy JavaScript phía server; Express.js là framework giúp xây dựng REST API nhanh gọn, nhẹ.



IV. CÁC CÔNG NGHỆ SỬ DỤNG

Database

- **MongoDB + Mongoose** — MongoDB là cơ sở dữ liệu NoSQL lưu dữ liệu dạng document (JSON-like); Mongoose là ODM giúp định nghĩa schema và thao tác với MongoDB dễ dàng hơn trong Node.js.



VS



IV. CÁC CÔNG NGHỆ SỬ DỤNG

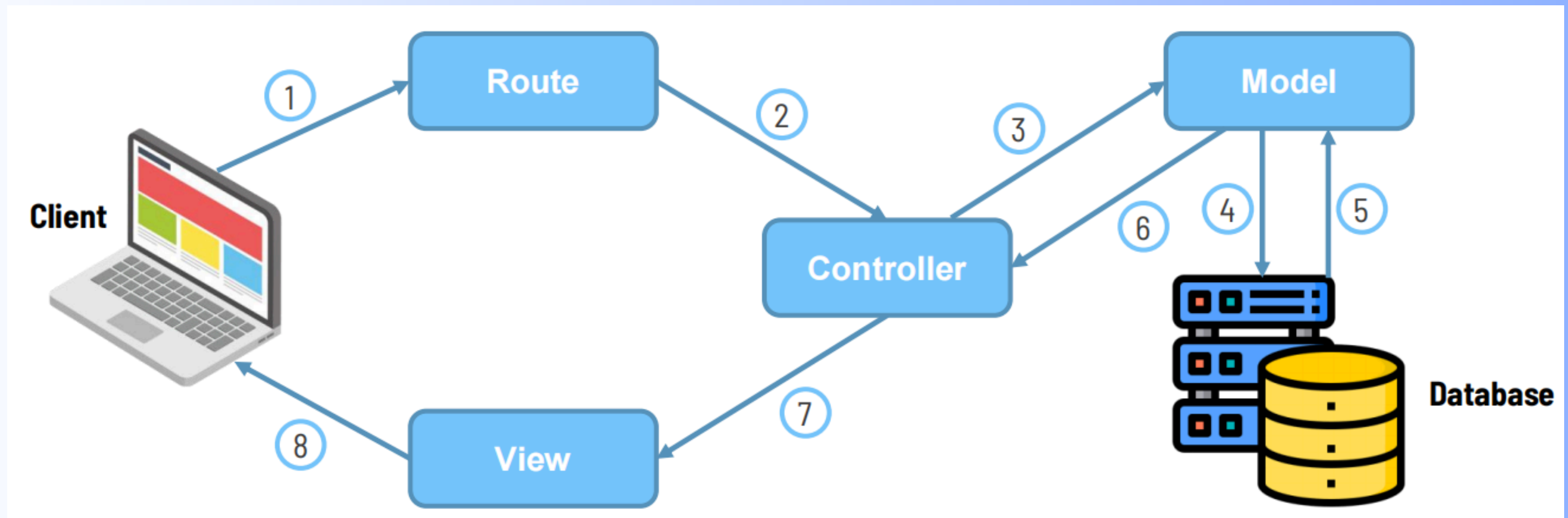
Các công nghệ hỗ trợ, bổ xung

- **JWT (JSON Web Token)** — Chuẩn xác thực stateless, sau khi đăng nhập server cấp token, client gửi kèm token ở mỗi request để chứng minh danh tính.
- **Bcrypt** — Thư viện mã hóa mật khẩu một chiều, đảm bảo mật khẩu không bao giờ lưu dạng plaintext trong database.
- **Joi** — Thư viện validate dữ liệu đầu vào, kiểm tra dữ liệu từ client đúng định dạng trước khi xử lý.
- **Multer + Cloudinary** — Multer xử lý upload file từ client; Cloudinary là dịch vụ lưu trữ ảnh/CV trên cloud, trả về URL để lưu vào DB.
- **Nodemon** — Tự động restart server mỗi khi có thay đổi code, không cần tắt bật thủ công.
- **ts-node** — Cho phép chạy TypeScript trực tiếp mà không cần compile ra JavaScript trước.
- **dotenv** — Đọc các biến môi trường từ file .env (API key, chuỗi kết nối DB...) giúp tách cấu hình khỏi code nguồn.

V. KHÁI NIỆM VỀ MÔ HÌNH MVC

MVC là viết tắt của Model - View – Controller

- Trong đó:
- Model (Mô hình): Dùng để kết nối với cơ sở dữ liệu
- View (Xem): Giao diện người dùng
- Controller (Điều khiển): Dùng để liên kết Model với View.



VI. CẤU TRÚC THƯ MỤC

```
SourceCode/
├── backend/           ← API Server (Port 4000)
│   ├── config/       ← Kết nối MongoDB
│   ├── controllers/  ← Xử lý logic nghiệp vụ
│   ├── middlewares/  ← Xác thực JWT
│   ├── models/       ← Schema dữ liệu (Mongoose)
│   ├── routes/       ← Định nghĩa API endpoints
│   ├── validates/    ← Kiểm tra dữ liệu đầu vào
│   ├── helpers/      ← Upload ảnh Cloundinary
│   └── index.ts       ← Khởi chạy server
└── frontend/         ← Giao diện (Port 3000)
    ├── src/
    │   ├── app/
    │   │   ├── (pages)/ ← Trang: home, job, company, search...
    │   │   └── components/ ← Header, Footer, Card dùng chung
    │   ├── hooks/      ← useAuth - kiểm tra đăng nhập
    │   └── config/     ← Biên cấu hình API
```

Model: job.model.ts, account-user.model.ts...

Định nghĩa cấu trúc dữ liệu, giao tiếp DB

View: Toàn bộ tầng Frontend (Next.js)

Hiển thị giao diện cho người dùng

Controller: auth.controller.ts, company.controller.ts...

Nhận request, xử lý logic, trả response

Luồng hoạt động:

Client (Next.js)

→ Route (/job, /company...)

→ Middleware (kiểm tra JWT)

→ Controller (xử lý logic)

→ Model (truy vấn MongoDB)

→ Trả JSON về Frontend

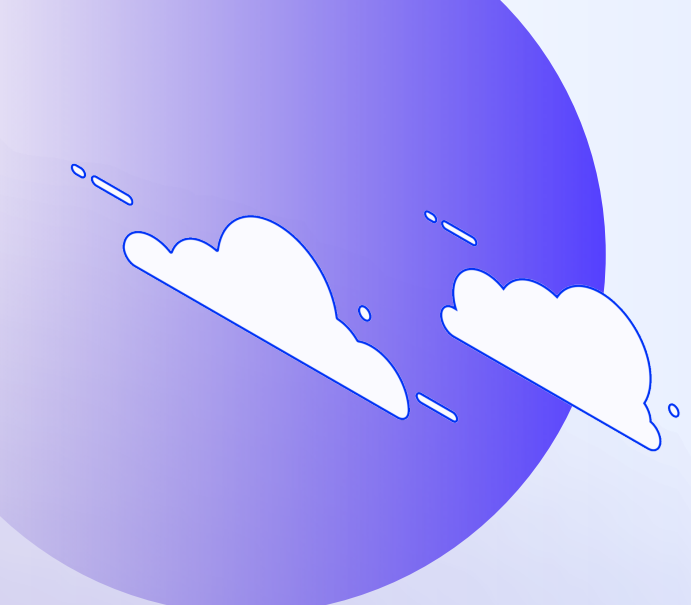
VII. TỔNG KẾT

Kết quả đạt được:

- Website hoạt động đầy đủ 2 vai trò: ứng viên & nhà tuyển dụng
- Xác thực bảo mật bằng JWT + Cookie
- Upload ảnh và CV lên cloud qua Cloudinary
- Tìm kiếm linh hoạt theo từ khóa và thành phố

Hướng phát triển:

- Gợi ý việc làm phù hợp theo AI
- Thông báo realtime khi có ứng viên mới
- Tối ưu SEO để tăng khả năng hiển thị



THANK YOU

FOR LISTENING !

bye~

